1. Clasifica las siguientes opciones en Alternativas Históricas Obsoletas, Estándar Nativo Universal o Dialectos/Superconjuntos con Transpilación:

- Adobe Flash (ActionScript)
- TypeScript
- Java Applets
- JavaScript (ECMAScript)
- JSX
- VBScript

Obsoletas	Estandar	Superconjuntos
Adobe Flash	JavaScript	TypeScript
Java Applets		JSX
VBScript		

---

2. Un equipo de desarrollo está debatiendo la arquitectura para una nueva aplicación web financiera.

- Explica dos ventajas técnicas de utilizar JavaScript nativo en el cliente para reducir el uso de recursos en el servidor.
- Define el concepto de Zero Plugins y menciona qué problema histórico soluciona frente a tecnologías antiguas.

Dos ventajas técnicas de JavaScript nativo en cliente para reducir recursos en el servidor:

- Ejecuta la lógica inmediata (validaciones, cálculos o interacción visual) en la máquina del usuario, liberando memoria y ciclos de CPU del servidor.
- Optimiza el ancho de banda al transferir únicamente datos ligeros (generalmente JSON) en lugar de recargar páginas HTML completas.

Concepto de Zero Plugins: Es la capacidad de ejecutarse e interpretarse de forma nativa por los motores de renderizado de los navegadores sin obligar al usuario a instalar extensiones o complementos externos. Soluciona la lentitud, la fragmentación y los graves agujeros de seguridad que generaban tecnologías antiguas como Adobe Flash o los Java Applets.

---

3. A partir de los requisitos de tres proyectos distintos, indica la tecnología recomendada (Vanilla JS, TypeScript o JSX) y justifica tu respuesta en base al tipo de proyecto y la fase de detección de errores:

- Proyecto A: Una plataforma empresarial bancaria con un equipo de 30 desarrolladores que requiere validación estática de datos en tiempo de desarrollo.

- Proyecto B: Un script ligero de 50 líneas para manipular el DOM e integrar un widget flotante en un sitio web estático.

- Proyecto C: Una interfaz de usuario interactiva basada en componentes reutilizables utilizando ReactJS.

- **Proyecto A (Plataforma bancaria, 30 desarrolladores, validación estática):** TypeScript. Está optimizado para proyectos corporativos de gran escala y ofrece tipado estático para cazar errores durante el desarrollo y la compilación.
- **Proyecto B (Script de 50 líneas para el DOM):** JavaScript Nativo (Vanilla JS). No requiere pasos intermedios de compilación y es la vía más rápida y ligera para scripts de tamaño pequeño o manipulación directa del DOM.
- **Proyecto C (Interfaz interactiva con ReactJS basada en componentes):** JSX. Es la extensión de sintaxis estándar en el ecosistema React que permite mezclar la lógica con el marcado de componentes reutilizables.

---

4. Define la relación entre ECMAScript y JavaScript. Explica qué papel desempeña un transpilador (como Babel) cuando se utiliza sintaxis moderna de ES6/ES2015 en entornos corporativos que deben mantener compatibilidad con navegadores antiguos.

- **Relación entre ECMAScript y JavaScript:** ECMAScript es la especificación técnica internacional (norma ECMA-262) que define cómo deben ser la sintaxis y el comportamiento del lenguaje. JavaScript es la implementación real de dicho estándar que ejecutan los motores de los navegadores.
- **Papel del transpilador (ej. Babel):** Traduce código moderno escrito en especificaciones actuales (como ES6/ES2015 o superiores) a versiones previas de JavaScript universalmente compatibles con navegadores antiguos.

---

5. Analiza la siguiente lista de nombres de variables. Indica cuáles son válidos y cuáles son inválidos según las reglas de JavaScript. Para los inválidos, especifica el motivo exacto de la infracción:

- 1erUsuario
- _totalFactura
- $elementoDOM
- precio-final
- class
- montoTotal2

- nombre usuario
- function
- 1erUsuario: Inválido. Un identificador no puede empezar por un dígito o número.
- _totalFactura: Válido. Puede comenzar por un guion bajo.
- $elementoDOM: Válido. Puede comenzar por el símbolo de dólar.
- precio-final: Inválido. Contiene un guion medio -, el cual es un operador aritmético (resta) y no un carácter permitido en identificadores.
- class: Inválido. Es una palabra reservada del lenguaje.
- montoTotal2: Válido. Empieza por una letra y contiene números al final, lo cual está permitido.
- nombre usuario: Inválido. Los identificadores no pueden contener espacios en blanco.
- function: Inválido. Es una palabra reservada del lenguaje.

---

6. Detección de Errores por Sensibilidad a Mayúsculas (Case-sensitive): El siguiente código JavaScript presenta múltiples errores sintácticos o de referencia debido al mal uso de mayúsculas y minúsculas. Identifica los 4 errores presentes y reescribe el fragmento corregido:

```
Function calcularDescuento(PrecioBase) {
  Const porcentaje = 0.15;
  If (PrecioBase > 100) {
    Alert("Descuento aplicado");
    Return PrecioBase * porcentaje;
  }
  return 0;
}

Console.log(calcularDescuento(150));
```

Los 4 errores presentes en las palabras reservadas o nativas son:

- **Function:** Debe escribirse en minúsculas (function).
- **Const:** Debe escribirse en minúsculas (const).
- **If y Return:** Deben ir en minúsculas (if, return).
- **Alert y Console:** Las funciones/objetos nativos deben escribirse en minúsculas (alert(), console.log()).

---

7. Aplicación de Convenciones Profesionales: Refactoriza los nombres de las siguientes entidades aplicando la convención de nomenclatura adecuada (camelCase, PascalCase, UPPER_SNAKE_CASE, $ o _):

- Una variable para almacenar el nombre del cliente actual.
- Una constante global inmutable para el número máximo de reintentos de conexión.
- Una clase que representa a un usuario registrado en el sistema.
- Una variable que almacena una referencia directa a un botón del DOM.
- Una propiedad de un objeto con intención de uso privado para guardar el token de sesión

- Variable para el nombre del cliente actual: `nombreClienteActual` (camelCase).
- Constante global inmutable para reintentos máximos: `MAX_REINTENTOS` o `MAX_REINTENTOS_CONEXION` (`UPPER_SNAKE_CASE`).
- Clase que representa un usuario registrado: `UsuarioRegistrado` (PascalCase).
- Variable que referencia un botón del DOM: `$boton` o `$botonEnviar` (prefijo `$`).
- Propiedad de objeto privada para guardar el token: `_tokenSesion` (prefijo `_`).

---

8. Análisis del Comportamiento del ASI (Inserción Automática de Punto y Coma): Observa el siguiente bloque de código:

```
function crearConfiguracion() {
  return
  {
    modo: "oscuro",
    puerto: 8080
  };
}

const config = crearConfiguracion();
console.log(config);
```

a) ¿Qué valor se imprimirá exactamente en la consola del navegador al ejecutar este script?

b) Explica paso a paso cómo interviene el mecanismo ASI del motor de JavaScript para producir dicho resultado.

c) Reescribe el código corregido aplicando las buenas prácticas marcadas en el temario

a) Valor impreso en consola: Se imprimirá `undefined`.

b) Intervención del ASI: Como la sentencia `return` va seguida inmediatamente de un salto de línea sin ninguna expresión en esa misma línea, el mecanismo ASI asume erróneamente que la sentencia ha finalizado y coloca un punto y coma automático (`return;`), ignorando el bloque del objeto posterior como retorno.

c) Solución con buenas prácticas: La llave de apertura `{` del objeto debe colocarse en la misma línea que la sentencia `return` (además de añadir punto y coma explícito al final de la declaración del objeto): `return {`.

---

9. Indica qué salida se obtiene en la consola al ejecutar de forma secuencial las siguientes líneas y explica el comportamiento del motor respecto a identificadores independientes:

```
let totalVentas = 500;
let TotalVentas = 1200;

console.log(totalVentas);
console.log(TotalVentas);
```

Salida obtenida: Primera línea: `500`

Segunda línea: `1200`

Comportamiento del motor: JavaScript es estrictamente case-sensitive (sensible a mayúsculas y minúsculas). Por ello, `totalVentas` y `TotalVentas` son tratados como dos identificadores completamente diferentes que apuntan a posiciones de memoria independientes.